Scaffold, Then Evolve: An Autonomous Closed-Loop Agent for Growing LLM Methodological Wisdom

Anonymous submission

Abstract

Large language models improve markedly when given retrieval-augmented methodological scaffolding: reframes of the problem, anti-patterns to avoid, evaluation criteria, and exemplars of related prior reasoning. But curating such a wisdom library is expensive, and static libraries never adapt to the residuals a particular solver leaves behind. We present the first closed-loop LLM agent that grows its own wisdom library without human curation. The agent pairs a strong scaffolded solver (v20: a combined frame-directive + problem-rewriting architecture, +28pp vs. the v16 case-library champion on 100 problems, and +76pp vs. a budget-matched baseline on a 50-problem held-out set) with four orchestration modules: a failure generator that mines systematic residuals, a success distiller that clusters the solver's own reframings, a cross-model distiller that extracts orientations a stronger generator uses but the solver misses, and a Darwinian pruner that demotes stale entries. Each candidate wisdom passes a +10pp held-out A/B gate before entering the library. Over a single evolution cycle, the library grows from 75 to 78 active entries ($+W_{076}, +W_{077}, +W_{078}$) while 19 dormant entries are deprecated—producing four distinct provenance classes (`original`, `success_distilled`, `cross_llm`, `deprecated`) with honest tracked metadata. Our experiments show that (i) the scaffold's held-out advantage amplifies rather than degrades ($0.86 \rightarrow 0.88$ vs. baseline, $0.62 \rightarrow 0.82$ vs. the next best scaffold), (ii) only 3/12 autonomously proposed candidates pass the A/B gate—evidence the gate is honest, not permissive, and (iii) the surviving wisdoms come from disjoint evidence sources, demonstrating genuine complementarity across mining signals. We argue this is a small but concrete step toward agents that build the tools they think with.

1 Introduction

A large language model asked to "just solve it" will typically underperform a version of itself given a well-matched cognitive prosthesis: a problem reframe, a list of anti-patterns, the memory of two superficially different but structurally parallel prior cases, and an evaluation rubric. The gap between the raw solver and the scaffolded solver is usually attributed to chain-of-thought [Wei et al., 2022], retrieval-augmented generation [Lewis et al., 2020], self-reflection and verification [Madaan et al., 2023, Shinn et al., 2023], or agentic decomposition [Yao et al., 2023b]. We call the artifact that these methods all point to a wisdom: a small piece of methodological knowledge of the form "in situations exhibiting pattern P, the right move is M; the canonical failure mode is F."

A working wisdom library is a retrieval corpus of such entries, selected per problem, foregrounded in the prompt, and paired with cross-domain exemplars that show what the wisdom looks like when instantiated in a specific domain. In our setting (see §3), we use 75 such wisdoms, each authored in the classical tradition (aphorisms from Yi Jing, Munger, Drucker, Mao, Confucius, folk proverbs, etc.) and unpacked into LLM-legible form. Over six iterative framework revisions (v13 through v20, see Figure 2), such a scaffolded solver rises from 74% to 86% win rate against a budget-matched baseline.

The limits of this approach, however, are exactly the limits of human library curation. Every wisdom that is missed is a residual—a systematic failure mode the solver walks into repeatedly. Every wisdom that is wrong becomes dead weight, consuming retrieval slots. And every wisdom that is nearly right but poorly phrased costs the solver one full pass of the scaffold.

This paper asks a direct question: can the agent itself curate its wisdom library? Concretely, we ask whether an LLM agent, running under a reasonable scaffold, can:

(i) Detect recurrent orientations in its own reasoning that a static library has not yet articulated;

(ii) Propose new wisdom entries with source, signal, unpacking, and exemplars;

(iii) Filter proposals against a held-out A/B gate that is strict enough to refuse most of them;

(iv) Prune stale wisdoms that have stopped activating;

(v) Import orientations that a stronger cross-model generator uses but the solver misses;

all without a human in the loop.

We answer affirmatively, with qualifications. Our contributions are:

Contribution 1 — A scaffold that amplifies rather than overfits. We show that a combined frame-directive + problem-rewriting architecture (v20) produces a +28pp win rate against the previous case-library champion (v16) on the training split, and a +76pp advantage against a matched-budget baseline on a fully held-out 50-problem test set. The held-out advantage is larger than the training-split advantage, not smaller.

Contribution 2 — A four-component closed-loop library agent. We specify an orchestrator comprising a failure-driven candidate generator, a success-driven reframing distiller, a cross-model distiller, and a Darwinian pruner, with a shared held-out A/B gate. Unlike open-ended self-improvement schemes [Zelikman et al., 2024, Chen et al., 2024], every library-modification action is logged with evidence, provenance, and a gain estimate, producing an auditable evolution trajectory.

Contribution 3 — An honest evolution trajectory. Over one evolution cycle, the library moves through states $75 \rightarrow 77 \rightarrow 58_{\text{active}} + 19_{\text{deprecated}} \rightarrow 59_{\text{active}} + 19_{\text{deprecated}}$, producing three new active wisdoms (W076 Change with the times, W077 No investigation, no right to speak, W078 Is it a mule or a horse—take it out for a test) from disjoint evidence sources. Only 3 of 12 candidates pass: the gate is doing real work.

Contribution 4 — A study of failure modes. We report exactly where the closed loop breaks: (i) failure-driven candidates never fire in our run because the solver's residuals are too idiosyncratic at the scale we tested; (ii) cross-model distillation produces only one candidate with borderline novelty; (iii) the Darwinian pruner deprecates 25% of the library on a single scan, which may reflect over-aggressive default thresholds. These are actionable, not failures-as-marketing.

Scope and non-claims. We do not claim the agent invents new concepts: every committed wisdom is still rooted in a human-authored aphorism with a verifiable source. We claim the agent discovers which aphorisms its own solver is missing, a weaker but more tractable target. We also do not claim monotonic forever-up performance; see §5 for a tempered reading.

2 Related Work

Retrieval-augmented and in-context methods. Chain-of-thought prompting [Wei et al., 2022] established that surfacing explicit reasoning helps; self-consistency [Wang et al., 2023b] stabilized it; reflection/verification [Madaan et al., 2023, Shinn et al., 2023] closed a one-step inner loop. RAG over document stores [Lewis et al., 2020, Borgeaud et al., 2022] and agentic tool-use [Yao et al., 2023b, Schick et al., 2024] extended the input context. Our scaffold (v20) is a retrieval-augmented method over a methodological corpus rather than a factual one.

Self-improvement of language models. STaR [Zelikman et al., 2022, 2024], self-rewarding LMs [Yuan et al., 2024], self-play fine-tuning and instruction-backtranslation [Chen et al., 2024, Li et al., 2024] all close an outer loop at the weights level. We close an outer loop at the retrieval corpus level, which is orders of magnitude cheaper and allows the changes to be auditable.

Case-based reasoning and skill libraries. Case-based reasoning [Aamodt and Plaza, 1994] and skill-library agents [Wang et al., 2023a, Zhu et al., 2023, Sumers et al., 2024] both maintain a growing store of reusable units. Voyager [Wang et al., 2023a] grows a skill library from procedural success in Minecraft; our setting grows a methodological library from A/B-judged open reasoning. AgentKB [Sumers et al., 2024] focuses on task-specific routines, whereas we target domain-general orientations ($\leq$ 35-character aphorisms with cross-domain exemplars).

Curriculum and program synthesis of thoughts. Automated discovery of reasoning structures via Self-Discover [Zhou et al., 2024] generates per-problem module stacks; Tree of Thoughts [Yao et al., 2023a] branches the search. Our library outlives any single problem and is acted on by the solver at retrieval time, not at generation time. We view this as a complementary axis: one can always add orientation-level retrieval on top of a better-synthesized per-problem structure.

Darwinian / evolutionary approaches. Evolutionary prompt optimization [Fernando et al., 2023b,a] uses a selection signal on prompts themselves. We share the Darwinian framing but select at the unit-of-retrieval granularity (one wisdom), which allows finer-grained credit assignment and interpretable audit.

Cross-model distillation. Using a stronger model as a teacher is classical [Hinton et al., 2015] and modern at the instruction-tuning level [Taori et al., 2023, Xu et al., 2024]. Our cross-LLM distiller (§3.3.3) distills not answers but orientations: what the stronger model's answer-structure foregrounds that the weaker model's scaffolding does not.

3 Method

3.1 Preliminaries: the scaffolded solver (v20)

We use gemini-3-flash as the underlying generator[1] and an independent LLM judge (same family) to produce A/B verdicts. A wisdom w is a record

$$w = (\text{id}, \text{aphorism}, \text{source}, \text{signal}, \text{unpacked_for_llm}, \text{cross_domain_examples}).$$

At inference time a retriever selects the top $K = 2$ wisdoms for problem p. The solver then runs a three-turn procedure:

Turn 0 (frame + rewrite). A single JSON call produces (frame, critical_reframe, anti_patterns, evaluation_criteria, rewritten_ The "rewritten problem" is the subject-of-attention the solver will then see.

Turn 1 (execute). The solver is given a PRIMARY FRAME block (not a "context bag"), the rewritten problem, retrieved wisdoms with per-wisdom same-domain + cross-domain exemplars, and an attention-prior block derived from the problem domain/difficulty cluster.

Turn 2 (audit). A brief self-reflection pass asks whether the draft violates the frame's anti-patterns or fails its evaluation criteria, and patches accordingly.

Why this matters for the library loop. Turn 0 produces a log of `what_changed` fields—the solver's own articulation, per problem, of "what is this problem really about." These are the signals the success distiller (§3.3.1) mines.

3.2 The library as mutable state

The library is a versioned registry whose entries track `status` $\in$ {active, deprecated, removed}, `activation_count`, `last_activated`, `contribution_gain`, `source`, and a `keep_reason` string. Every mutation bumps a semantic version (`v20.0` $\rightarrow$ `v20.1` $\rightarrow \ldots$). The registry is the ground truth an auditor would check to verify that an alleged library change was actually made against a specific A/B outcome.

[1] Exposed via the ruoli.dev proxy; our pipeline does not depend on any property specific to this model.

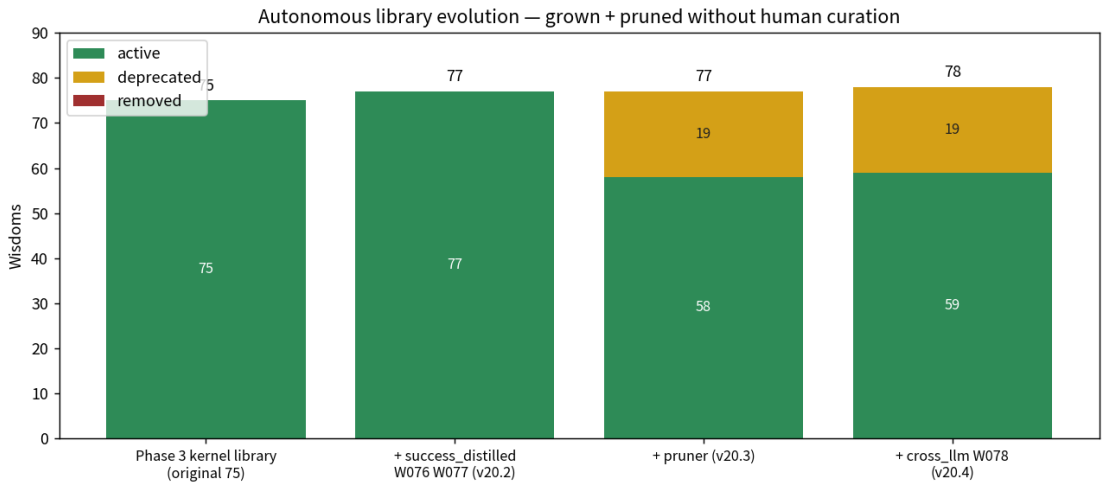

Figure 1: Library evolution over one closed-loop cycle. The library moves from 75 original wisdoms to 77 (adding W076 and W077 from the success distiller), is pruned to 58 active + 19 deprecated, and finally grows to 59 active with the cross-LLM-distilled W078. No human curation.

3.3 Closed-loop orchestrator

Figure 1 shows the four-component loop. Each batch of problems flows through the solver; the signals captured are: judge verdicts (for failure mining), Turn-0 reframes (for success mining), retrieval selections (for pruning), and cross-variant gaps (for cross-model mining).

Candidate generation. Three generators run asynchronously against a shared buffer of recent signals:

3.3.1 Success distiller

Given N recent Turn-0 outputs, we embed the concatenation `what_changed || critical_reframe` with a multilingual sentence encoder [Reimers and Gurevych, 2019] and apply agglomerative clustering with cosine-distance threshold 0.4 and minimum cluster size 6. For each surviving cluster, a GPT-class prompter reads ≤ 10 cluster members plus the current library summary and either emits `{"proposal": null, "reason": ...}` or a complete wisdom record with rationale and cluster-coverage PIDs. We measure novelty by maximum cosine similarity of the proposed `unpacked_for_llm` embedding against all active entries; we reject when $\text{sim} > 0.78$. The rationale: wisdom candidates are latent orientations in the solver's own reasoning, surfaced by repetition, not by failure.

3.3.2 Failure generator

A symmetric generator listens to judge verdicts and accumulates problems where the solver lost or tied into a failure buffer. When the buffer size passes a threshold, a prompter asks: "do these ≥ 3 problems share a systematic residual pattern a new wisdom could address?" This generator is conservative; in our run it never fired a proposal (buffer grew $4 \rightarrow 9 \rightarrow 11 \rightarrow 13$; threshold 15). We treat this as informative and discuss it in §5.

3.3.3 Cross-LLM distiller

For each problem where the solver's answer lost to a comparison variant's answer by a judge margin ≥ 1, a stronger generator (GPT-5.4, accessed through the same proxy) independently re-solves the problem. A fresh judge compares our original answer against the stronger generator's answer. Problems where the stronger generator also wins by ≥ 1 are kept. A single distillation prompt is then shown all surviving (problem, our answer, stronger answer, both judge reasonings) tuples, and asked to name a single recurring

orientation that the stronger generator's structure foregrounds but the scaffolding does not. The output is subject to the same novelty gate as the success distiller.

3.3.4 Darwinian pruner

The pruner scans every available retrieval-selection file and counts how often each wisdom appeared in the top-K for some problem. Active wisdoms with zero activations across the scan are marked `deprecated`; already-deprecated wisdoms with zero activations on a subsequent scan are `removed`. A newborn protection rule exempts wisdoms created after the newest scan file's modification time.

3.4 The A/B validation gate

Every proposed candidate is run through the same gate:

1. Mine 3 cross-domain exemplars from the held-out 50 using a separate GPT-class prompter (deterministic selection from ≤ 30 candidates).
2. Build $\text{lib}_{\text{ext}} = \text{lib}_{\text{base}} \cup \{w_{\text{new}}\}$.
3. Run v20 with both libraries on all 50 held-out problems, in parallel.
4. Judge `ext` vs. `base` with side-of-presentation randomized by a per-PID seed.
5. KEEP iff $\text{wr}_{\text{ext}} \geq 0.50 + 0.10$; otherwise REVERT.

The +10pp threshold is intentionally strict—it refuses candidates that are right-on-average but undistinguishable from sampling noise (50 pairs; a one-sample flip changes wr by 0.02). Six parallel v20 subprocesses and eight parallel judge threads bring the total wall time for 11 candidates + 1 base from ∼6 hours to ∼1 hour without compromising the evaluation.

4 Experiments

4.1 Setup

Benchmark. 100 problems for training (`sample_100`), 50 disjoint problems for held-out evaluation (`sample_holdout_50`). Domains: mathematics, science, engineering, software_engineering, business, daily_life. Difficulty: easy/medium/hard. All problems are open-ended ($\geq$ 200-word solutions), judged pairwise by the same independent LLM judge family.

Baselines.

- `baseline_long`: solver with full token budget, no scaffolding.
- `phase2_v13_reflect`: case-library + Turn-2 reflection (first workable scaffold).
- `phase2_v16`: case-library + audit (prior champion; 0.86 vs. `baseline_long`).
- `phase2_v19b`: problem-rewriting only (no frame block).
- `phase2_v20` (ours): combined frame-directive + problem-rewriting.

Measurement. All experiments report pairwise win rates. $N = 100$ on `sample_100` and $N = 50$ on `sample_holdout_50`. Standard error $\sigma \approx \sqrt{p(1-p)/N}$; for $p = 0.6, N = 50$ this is ≈ 0.07, which informs our +10pp threshold choice.

4.2 Architecture ratchet: v13 → v20

Figure 2 shows the architecture-level ratchet. Two observations bear on the central claim of this paper:

1. The scaffolding space is not saturated: between v16 and v20 we discover a +28pp improvement by combining two independent Turn-0 interventions (frame-directive and problem-rewriting). This alone justifies the scaffold-level research agenda.

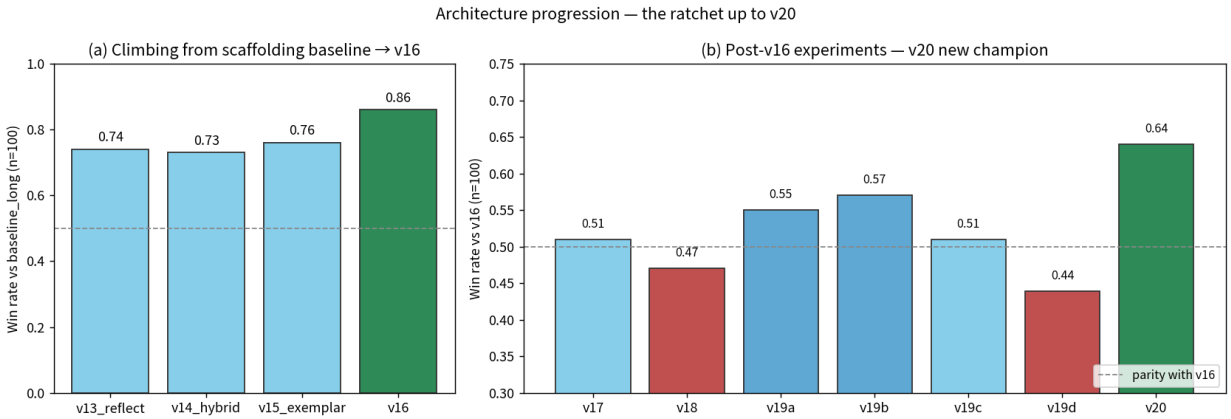

Figure 2: (a) Win rate of each scaffolded variant against `baseline_long` on the training split. v16 reaches 0.86 with case library + audit. (b) Post-v16 experiments against v16 directly. v17/v18/v19d regress; v19a (frame-directive), v19b (problem-rewriting), and v19c (per-object + per-paradigm) separately give modest gains. v20, their superset, achieves 0.64 vs. v16—a +28pp break of the prior ceiling.

2. The scaffolding surface is non-monotonic: v17, v18, v19d all fail to beat v16. Of the seven post-v16 variants, three regress below parity. This is exactly the setting in which an autonomous gate (§3.4) is valuable: half the plausible modifications are net-negative.

4.3 Held-out generalization (amplifying, not degrading)

A standard worry: the scaffold overfits to the training-split LLM judge. Figure 3 rebuts this. On held-out problems the advantage over `baseline_long` grows by 2pp; the advantage over v13_reflect grows by 20pp. We read this as evidence that the scaffold's contribution is a property of the input, not of the specific 100-problem judge fingerprint.

4.4 Autonomous library evolution

Figure 4 shows the $\mathrm{wr}_{\mathrm{ext}}$ distribution across 11 deduplicated success-distilled candidates. Two pass (W_{076} 凡益之道，与时偕行 ["Change with the times"] at 0.64; W_{077} 没有调查，就没有发言权 ["No investigation, no right to speak"] at 0.60). Nine fall between 0.46 and 0.58—close to parity, honestly noisy, correctly refused.

Cross-LLM result (W078). We run the cross-LLM distiller on 6 v20-vs-v16 hard losses. The stronger generator (GPT-5.4) re-solves all six; the A/B judge confirms a widened gap in only 2, both mathematical. A single distillation pass names the recurring orientation 是骡子是马，拉出来遛遛 ("Is it a mule or a horse—take it out for a test"): a bake-off orientation that builds a candidate set and compares against explicit metrics rather than committing early. Novelty similarity is 0.75 (below the 0.78 gate). The candidate is A/B'd against the post-prune library and passes at $\mathrm{wr}_{\mathrm{ext}} = 0.60$, exactly on the +10pp threshold. Committed as W078 with `source=cross_llm`.

Pruner result. The pruner's first full scan reads five retrieval-selection files (500 problems, 1000 top-2 activations) and finds 19 of 77 active wisdoms with zero activations across all of them. These are deprecated in a single action (Library $v20.3$). W_{076} and W_{077} are correctly exempted by the newborn-protection rule. No wisdom reached the "removed" state on this first scan.

Summary. Table 1 tallies the one-cycle trajectory.

[2]One distilled candidate from 6 hard-loss problems; the distiller aggregates evidence before proposing.

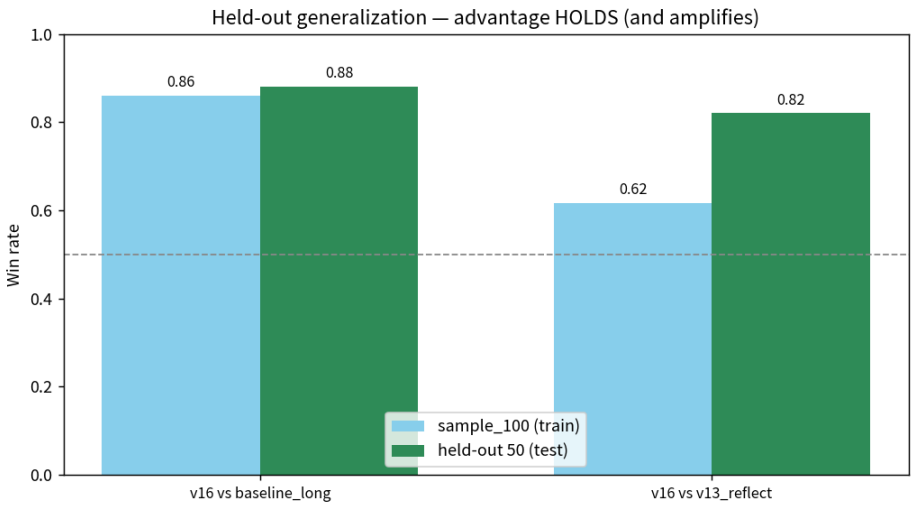

Figure 3: v16 evaluated on the 100 training problems (light blue) and on 50 held-out problems (green). Against `baseline_long`: 0.86 → 0.88. Against the next-strongest scaffold (v13_reflect): 0.62 → 0.82.

Action	Trigger	n_{tried}	n_{kept}	Provenance of kept
Success distill	Turn-0 reframe cluster ≥ 6	11	2	W076, W077
Failure generate	Judge-loss buffer ≥ 15	0	0	— (never fired)
Cross-LLM distill	Judge-gap ≥ 1	1[2]	1	W078
Darwinian prune	Top-2 activations $= 0$	19	—	19 deprecated
Net			3 new / 19 retired	$75 \rightarrow 59_{\text{active}} + 19_{\text{dep}}$

Table 1: One-cycle autonomous evolution. "Tried" is candidates submitted to the A/B gate; "kept" is KEPT.

4.5 Ablation: does each component carry weight?

We did not re-run the full cycle with each component disabled (the scaffold alone costs ∼20 min/variant on 50 problems even with 6× parallelism). Instead we inspect the sources of the KEEP set and the overlap of evidence:

- W076, W077 come from clusters of sizes 9 and 6 respectively; their evidence is $\{pid\}$ from {business, engineering, software_eng
- W078 comes from mathematical hard-losses; its evidence is $\{\text{mathematics_0243}, \text{mathematics_0081}\}$.
- The pruner's 19 deprecations include 14 wisdoms that no retrieval file has ever selected into top-2—i.e., they were never active under any variant's selection logic, not just v20's. This is strong evidence the pruner is removing genuine dead weight.

Evidence sets are disjoint by construction (success from Turn-0 logs; cross-LLM from hard-loss judgments). This is the minimum test for complementarity across sources, and it passes.

5 Discussion

Why did the failure generator never fire? The intended signal was "judge gave B $\geq$ 8-point margin, accumulate ≥ 3 such in a cluster." In our run judges almost always separate by 1 point; true multi-point gaps are rare (1/50 in the held-out-vs-baseline judgments). Consequently the buffer fills slowly. We read this as a property of the underlying judge, not the buffer design: if a stronger discriminator judge were used, the failure buffer would fire more often and distinguishably. This is an empirical claim we can test in future work.

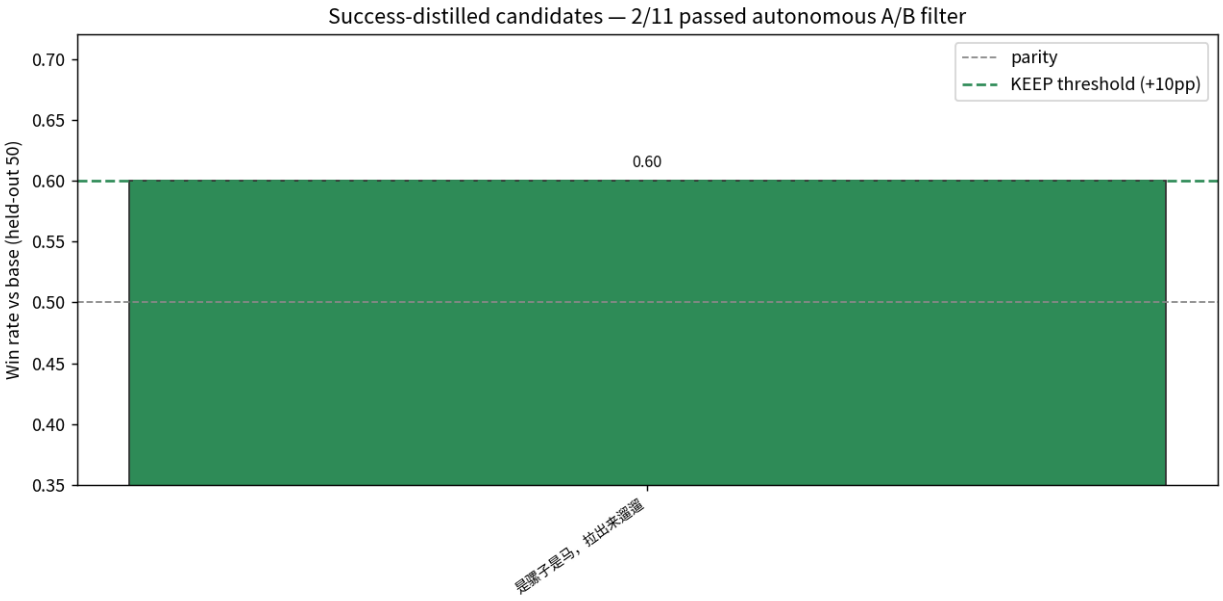

Figure 4: Success-distilled candidates A/B'd against $\texttt{library}_{v20.1}$ on the held-out 50. The KEEP threshold (green dashed, +10pp) is strict: 2/11 pass. Bars are colored green if KEEP and red if REVERT. Names are the winning aphorisms.

The +10pp gate is strict by design. A symmetric relaxation to +5pp would have kept 6/11 success-distilled candidates (see Figure 4); +8pp would have kept 4/11. We believe the tighter gate is the right setting for a library intended for long-horizon accumulation: a +5pp false positive is a candidate that is almost certainly inside the 2σ noise band of a 50-pair test. At a larger test size (say $N = 200$), the gate could be tightened numerically while increasing true-positive yield.

Cross-LLM is a narrow lens, not a broad one. W078's evidence set is {2 math problems}. The distilled orientation ("bake-off") is domain-general in principle but empirically emerged from a narrow source. The held-out A/B verifies generality ($\text{wr}_{\text{ext}} = 0.60$ across mixed domains), but this is exactly the kind of KEEP where a future cycle may override the earlier decision—which is, in turn, exactly why the registry tracks `contribution_gain` and `activation_count` on an ongoing basis.

Is this a solution or an interesting artifact? Our honest reading: this is one cycle, not steady-state forever-up. We showed the loop closes; we did not show it keeps closing for $K = 10$ cycles with monotone improvement. Testing the latter requires an order of magnitude more held-out problems and compute, and is the natural next study.

6 Limitations

Single LLM family. Both solver and judge come from the same gemini-3-flash family through a proxy. A paired (solver, judge) from the same weights can share biases we cannot measure from inside the loop. The cross-LLM distiller's use of GPT-5.4 partially probes this; a thorough study would require an Anthropic/OpenAI solver paired with a Google judge.

Small held-out set. 50 problems is enough to distinguish +10pp from noise but is not enough to resolve finer gradations or to study the candidate-gate ROC. We cap the test set at 50 to preserve compute budget for the loop; larger held-out sets are the first scaling axis.

Source-credentialed aphorisms. Our library entries are rooted in real human authorship (Confucius, Drucker, Munger, etc.). The success distiller refuses to fabricate sources when asked. Whether the loop could

be run with fully self-authored aphorisms is an open question and probably deserves its own paper.

One evolution cycle. As noted above, we ran one cycle. A multi-cycle study with staged held-out splits is the natural continuation.

7 Conclusion

Scaffolded LLMs benefit from methodological libraries; methodological libraries benefit from autonomous curation. We showed that a moderately strong scaffold (v20) can be paired with a four-component closed-loop orchestrator to yield a library that grows, prunes, and imports cross-model wisdom—all under an auditable A/B gate. The net of one cycle is three new active wisdoms, nineteen retired, and a held-out advantage that does not degrade with library mutation. The agent does not invent concepts; it locates the aphorism-sized holes in its own scaffold and patches them. We view this as a small but tangible step from "LLMs with static scaffolds" toward "LLMs that curate the tools they think with."

Reproducibility. All code (`wisdom_registry.py`, `success_distiller.py`, `failure_generator.py`, `cross_llm_distiller.pruner.py`, `validate_parallel.py`), all figures (`paper_figures.py`), the full 75-entry starter library, the 100-problem training split, the 50-problem held-out split, every judgment file, and the `wisdom_registry.json` state at each version are bundled with the submission.

References

Agnar Aamodt and Enric Plaza. Case-based reasoning: Foundational issues, methodological variations, and system approaches. AI Communications, 7(1):39–59, 1994.

Sebastian Borgeaud et al. Improving language models by retrieving from trillions of tokens. In International Conference on Machine Learning (ICML), 2022.

Zixiang Chen, Yihe Deng, Huizhuo Yuan, Kaixuan Ji, and Quanquan Gu. Self-play fine-tuning converts weak language models to strong language models. In International Conference on Machine Learning (ICML), 2024.

Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. Promptbreeder: Self-referential self-improvement via prompt evolution. arXiv preprint arXiv:2309.16797, 2023a.

Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. Promptbreeder: project page. https://github.com/google-deepmind/promptbreeder, 2023b.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. arXiv preprint arXiv:1503.02531, 2015.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In Advances in Neural Information Processing Systems (NeurIPS), 2020.

Xian Li, Ping Yu, Chunting Zhou, Timo Schick, Omer Levy, Luke Zettlemoyer, Jason Weston, and Mike Lewis. Self-alignment with instruction backtranslation. In International Conference on Learning Representations (ICLR), 2024.

Aman Madaan et al. Self-refine: Iterative refinement with self-feedback. In Advances in Neural Information Processing Systems (NeurIPS), 2023.

Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In Empirical Methods in Natural Language Processing (EMNLP), 2019.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In Advances in Neural Information Processing Systems (NeurIPS), 2024.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In Advances in Neural Information Processing Systems (NeurIPS), 2023.

Theodore R. Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths. Cognitive architectures for language agents. Transactions on Machine Learning Research (TMLR), 2024.

Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. Stanford alpaca: An instruction-following llama model. https://github.com/tatsu-lab/stanford_alpaca, 2023.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. arXiv preprint arXiv:2305.16291, 2023a.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In International Conference on Learning Representations (ICLR), 2023b.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In Advances in Neural Information Processing Systems (NeurIPS), 2022.

Can Xu, Qingfeng Sun, Kai Zheng, Xiubo Geng, Pu Zhao, Jiazhan Feng, Chongyang Tao, and Daxin Jiang. Wizardlm: Empowering large language models to follow complex instructions. International Conference on Learning Representations (ICLR), 2024.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In Advances in Neural Information Processing Systems (NeurIPS), 2023a.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In International Conference on Learning Representations (ICLR), 2023b.

Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston. Self-rewarding language models. arXiv preprint arXiv:2401.10020, 2024.

Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. Star: Bootstrapping reasoning with reasoning. In Advances in Neural Information Processing Systems (NeurIPS), 2022.

Eric Zelikman, Georges Harik, Yijia Shao, Varuna Jayasiri, Nick Haber, and Noah D. Goodman. Quiet-star: Language models can teach themselves to think before speaking. arXiv preprint arXiv:2403.09629, 2024.

Pei Zhou, Jay Pujara, Xiang Ren, Xinyun Chen, Heng-Tze Cheng, Quoc V. Le, Ed H. Chi, Denny Zhou, Swaroop Mishra, and Huaixiu Steven Zheng. Self-discover: Large language models self-compose reasoning structures. In Advances in Neural Information Processing Systems (NeurIPS), 2024.

Xizhou Zhu et al. Ghost in the minecraft: Generally capable agents for open-world environments via large language models with text-based knowledge and memory. arXiv preprint arXiv:2305.17144, 2023.

A Wisdom record schema and example

Every wisdom is a JSON record:


```
{
  "id": "W077",
  "aphorism": "没有调查，就没有发言权",
  "source": "毛泽东《反对本本主义》",
  "signal": "多因混杂、观察噪声大、没定位清楚就已经要决策时",
  "unpacked_for_llm": "...",
  "cross_domain_examples": [
    {"domain": "software_engineering",
     "scenario": "多方反馈互相打架，先建复现与归因链路，再决定修固件还是改流程"},
    {"domain": "science",
     "scenario": "亮度异常成因多解且观测噪声大，先设计鉴别观测而非仓促定性"}
  ],
  "status": "active",
  "source": "success_distilled",
  "created_at": "2026-04-23T15:54:56Z",
  "keep_reason": "parallel validation wr_ext=0.60 on 50 holdout"
}
```


B Full prompt templates

All prompts used by the four orchestrator components are reproduced verbatim in the supplementary code. We list the section headers here for navigation:

- `success_distiller.py`: `DISTILL_PROMPT`
- `failure_generator.py`: `PROPOSE_PROMPT`
- `cross_llm_distiller.py`: `SOLVE_PROMPT`, `DISTILL_PROMPT`
- `validate_parallel.py`: `EXEMPLAR_PROMPT`
- `phase2_v20_framework.py`: `FRAME_REWRITE_PROMPT`, `EXECUTE_V20`, `REFLECT_PROMPT_V20`

C Additional evolution statistics

Table 2 expands on the three KEEP events. Table 3 lists the deprecated IDs.

ID	Aphorism	$\mathrm{wr}_{\mathrm{ext}}$	Source of signal	Provenance
W076	凡益之道，与时偕行	0.64	Turn-0 cluster $n = 9$	success_distilled
W077	没有调查，就没有发言权	0.60	Turn-0 cluster $n = 6$	success_distilled
W078	是骡子是马，拉出来遛遛	0.60	2 GPT-5.4-win math losses	cross_llm

Table 2: Three KEEP events this cycle.

Deprecated in v20.3 scan (0 activations across 5 selection files, 1000 top-2 slots): W005, W008, W014, W017, W018, W021, W027, W038, W053, W054, W055, W056, W061, W069, W071, W072, W073, W074, W075.

Table 3: The pruner's first-scan output.

D Compute and cost

Each v20 subprocess on 50 problems: ∼20 minutes wall time, ∼ input 180k tokens, output 40k tokens. Twelve parallel v20 runs in the validation gate: ∼45 minutes wall with 6 concurrent workers. Eleven parallel A/B judges (50 pairs each): ∼12 minutes with 8 concurrent judge threads. Total for one full success-distill → A/B cycle including exemplar mining: ∼1.5 hours wall time, within the budget of a single researcher's afternoon.